

PRD_15

Unified Module Logic, Domain Ownership and Cross-Module Interaction

Construction OS

Engineering specification for making every module operate as one connected system without duplicated business logic, conflicting records, or insecure cross-module access.

Core principle: Every record has one source of truth. Every action has one authorization policy. Every cross-module consequence uses an event. Every important change creates an audit entry.

0. Document Control

Field	Value
Document	PRD_15 - Unified Module Logic, Domain Ownership and Cross-Module Interaction
Purpose	Define common logic for Projects, Tasks, Contracts, Finance, Clients, Contractors, Documents, HR, Calendar and Platform Administration.
Audience	Product, Full Stack Engineering, Database, QA, Security and DevOps
Precedence	This PRD provides engineering rules that apply across all feature PRDs. Feature-specific business decisions remain valid unless they conflict with data ownership, authorization or audit requirements defined here.

Executive rule

Modules may display each other's data, but they must not create independent copies of another module's source records.

1. Table of Contents

2. Executive Summary
3. Problems This PRD Solves
4. Common Action Pipeline
5. Domain Ownership and Source of Truth
6. Profile and Dashboard Aggregation
7. Central Authorization and Record Access
8. Domain Events and Cross-Module Interaction
9. Controlled Record Lifecycles
10. Financial Truth and Reconciliation
11. Universal Linking Model
12. Documents, Comments, Tasks and Activity Timeline
13. Idempotency and Transaction Safety
14. Service Decomposition and Code Structure
15. Read Models, Search and Reporting
16. Audit and Observability
17. API and Data Model Requirements
18. Migration from Current Repository
19. Acceptance Criteria
20. Full Stack Implementation Checklist

2. Executive Summary

Construction OS is becoming a deep enterprise platform. Its strength is the connection between projects, contracts, clients, contractors, finance, HR, documents, units, tasks and platform governance. Its main architectural risk is allowing each module to develop its own permission logic, duplicated records, status rules and cross-module side effects.

PRD_15 establishes one common operating model for every module. It does not reduce functionality. It makes the functionality predictable, secure and scalable.

- One source of truth: each business fact belongs to one domain.
- One authorization path: every backend action evaluates the same policy context.
- One event mechanism: modules communicate through versioned domain events.
- One audit mechanism: all important actions produce structured audit records.
- One task, document, comment and notification infrastructure: shared platform engines, not duplicate implementations.
- One user experience language: consistent statuses, actions, timelines and linked records.

3. Problems This PRD Solves

- The same value may be stored in several modules and become inconsistent.
- Frontend visibility may be mistaken for backend authorization.
- Modules may write directly into each other's tables.
- Tenant membership may be incorrectly treated as company or record access.
- Status fields may be changed without valid workflow transitions.
- Dashboards may query restricted data and hide it only in the interface.
- Large services may contain unrelated business logic and become difficult to test.
- Repeated requests may create duplicate payments, tasks, approvals or notifications.
- Audit logs may be incomplete because each module records activity differently.

The objective is not to create microservices immediately. The objective is to create clear domain boundaries inside the current codebase so the system can later scale safely.

4. Common Action Pipeline

Every important command must follow the same ordered pipeline:

User action -> active scope -> authentication -> authorization -> input validation -> ownership validation -> transaction -> audit -> domain event -> notifications/read models

MOD-001 - No page, component or server action may write directly to Prisma without passing through an application service or command handler.

MOD-002 - The active tenant, company, role and record context must be resolved before authorization.

MOD-003 - All referenced IDs must be validated as belonging to the correct tenant and company before a write.

MOD-004 - Database changes that must succeed together must execute in one transaction.

MOD-005 - Audit and event records must be committed consistently with the business transaction.

MOD-006 - Notification delivery may be asynchronous, but the source event must be durable.

5. Domain Ownership and Source of Truth

Domain	Owns	May be displayed elsewhere
Projects	Project identity, members, progress, milestones	Dashboards, contracts, client pages, task pages
Contracts	Legal agreement, parties, revisions, approvals, status	Client, contractor, project and finance pages
Finance	Approved values, invoices, payment records, balances, reconciliation	Client, contractor, project and executive summaries
Clients	Client identity, contacts and relationships	Contracts, units, sales and finance pages
Contractors	Contractor identity, profile, qualifications and evaluations	Contracts, procurement, projects and finance
Units / Architecture	Unit identity, plans, drawings, design comments and revisions	Client, project and sales pages
Tasks	Responsibility, deadlines, status, approval and completion	All linked source records
Documents	File object, revision, official status, access and retention	Any linked record
HR	Employment, leave, attendance, salary and private records	Authorized calendars and management views
Platform Admin	Tenants, features, technical settings, recovery and support access	Platform governance panels

MOD-007 - Every business record must declare one owning domain.

MOD-008 - A module may reference another domain record by ID but must not maintain a competing copy of the source value.

MOD-009 - Derived summaries must be clearly identified as read models and must be rebuildable from source records.

MOD-010 - Only the owning domain may approve or mutate its authoritative fields.

6. Profile and Dashboard Aggregation

Profiles are composed views, not separate databases.

A contractor profile may show contractor identity, clickable contracts, finance totals, projects, documents, tasks and evaluations. Each section reads from its owning module. The contractor profile does not create a second contract or payment record.

The same rule applies to Client, Project, Unit, Employee and Company pages.

- A summary endpoint may combine authorized data from several domains.
- Each field must retain its source module and permission policy.
- Clicking a summary item opens the original record.
- Restricted data must not be fetched when the viewer lacks access.
- Profile totals must be calculated from authoritative records or approved read models.

7. Central Authorization and Record Access

The existing role matrix answers whether a user may enter a module. A central policy engine must answer whether the user may perform an action on a specific record.

Required policy context:

- tenant and active company
- active role and department
- record owner and source domain
- project and contract involvement
- explicit grants or revocations
- parent/child company context
- temporary Platform Admin support access
- sensitive-data classification

MOD-011 - Backend authorization is authoritative; hiding a button is never sufficient.

MOD-012 - Role claims in long-lived JWTs must not remain valid after demotion or revocation.

MOD-013 - Every command and query must use the central policy service.

MOD-014 - Permission checks must evaluate the exact record, not only the module.

MOD-015 - Restricted values must be excluded at query level rather than loaded and hidden.

8. Domain Events and Cross-Module Interaction

Modules communicate through versioned domain events rather than direct table mutation.

Example: Contract Approved

- Contract module commits the approved contract revision.
- The transaction creates a durable ContractApproved event.
- Finance creates or updates the financial structure.
- Project and profile read models update.
- Document module locks or marks the approved revision as official.
- Notifications are delivered to authorized users.

Example: Finance tags Architect in a contract comment

- Comment remains owned by the Contract/Comment domain.
- Task module creates one Architect task linked to the comment, contract, client and unit.
- Architect completes the task.
- Finance reviews and formally closes it.
- Task and comment show synchronized status through links and events.

MOD-016 - Every event must contain event_id, type, version, tenant_id, company_id, actor_id, source record, timestamp and correlation_id.

MOD-017 - Consumers must be idempotent.

MOD-018 - Use a transactional outbox so a committed business action cannot lose its event.

MOD-019 - Event consumers may update read models or create owned records but may not silently rewrite another domain's source record.

9. Controlled Record Lifecycles

Statuses must be governed by explicit state transitions, not editable free text.

Record	Example lifecycle
Contract	Draft -> Under Review -> Approved -> Active -> Completed / Terminated
Task	Draft -> Assigned -> In Progress -> Ready for Review -> Completed
Payment	Draft -> Confirmed -> Posted -> Reversed
Document	Draft -> Submitted -> Approved / Rejected -> Superseded / Archived
Red Flag	Open -> Under Review -> Closed

MOD-020 - Allowed transitions must be defined in backend code and tested.

MOD-021 - Transition permission, required fields and side effects must be validated together.

MOD-022 - Invalid transitions must be rejected even when the API is called directly.

MOD-023 - Every transition creates an audit entry and, where relevant, a domain event.

10. Financial Truth and Reconciliation

Finance retains authority over approved values while the system independently validates arithmetic.

- Finance manually enters and approves contract values.
- Every payment is stored as an individual payment record.
- Payment records include date, amount, reference, method and proof document.
- Client and contractor profiles display totals from Finance source records.
- Manual paid and remaining values may be preserved when required by confirmed PRDs.
- The system calculates expected totals and shows reconciliation warnings without silently overwriting Finance-approved values.
- All money fields use database Decimal, never Float.

MOD-024 - Summary totals must not be directly editable outside Finance-owned commands.

MOD-025 - Every manual override must preserve previous values and actor attribution.

MOD-026 - Reconciliation warnings must identify the mathematical difference and affected records.

MOD-027 - Payment posting and reversal must be transactional and idempotent.

11. Universal Linking Model

Tasks, comments, documents, meetings, flags and notifications must use a controlled linking model to connect to supported source records.

- Project
- Contract
- Client
- Contractor
- Unit

- Payment
- Invoice
- Document
- Employee
- Meeting or inspection

A generic link must store source type, source ID, tenant, company and relationship purpose. The backend must verify the target exists and belongs to the permitted scope.

MOD-028 - Generic links must use a controlled type registry rather than arbitrary strings.

MOD-029 - Cross-tenant and unauthorized cross-company links must be rejected.

MOD-030 - Deleting or archiving a source record must preserve link history where audit requirements demand it.

MOD-031 - Linked-record permissions do not automatically grant access to the full source module.

12. Documents, Comments, Tasks and Activity Timeline

Shared engines

- Task engine: one implementation for assignments, deadlines, approvals, completion, reopening and escalations.
- Comment engine: one implementation for mentions, attachments, visibility, linking and conversion to tasks.
- Document engine: one implementation for file storage, revision, official status, access and retention.
- Notification engine: one implementation for recipients, channels, preferences, delivery and unread state.
- Audit/activity engine: one implementation for immutable history and page timelines.

Every major page should show a standardized activity timeline derived from audit and domain events. Modules must not create unrelated timeline tables with incompatible formats.

13. Idempotency and Transaction Safety

Repeated requests must not create duplicate business effects.

- Payment posting
- Invoice creation
- Task creation from a comment
- Contract activation
- Approval submission
- Notification delivery
- External webhook processing

MOD-032 - Client requests that create important records must accept or generate an idempotency key.

MOD-033 - Unique database constraints must enforce business uniqueness where possible.

MOD-034 - Retries must return the original successful result rather than create another record.

MOD-035 - Partial failures must be recoverable through durable events and retryable background jobs.

14. Service Decomposition and Code Structure

The current repository should remain a modular monolith initially, but internal boundaries must become explicit.

Example decomposition for Task orchestration:

- task-command.service
- task-assignment.service
- task-approval.service
- task-inspection.service
- task-escalation.service
- task-completion.service
- task-event.publisher
- task-query.service
- task-policy

Application services may coordinate domains. Domain services own business rules. Repositories own persistence. UI components must not contain authoritative business logic.

15. Read Models, Search and Reporting

Dashboards and profiles should use permission-safe read models rather than joining every operational table on every request.

- Executive Project Summary
- Finance Exposure Summary
- Client Account Summary
- Contractor Payable Summary
- Personal Work Summary
- Group Consolidated Summary

MOD-036 - Read models must identify their source events and rebuild strategy.

MOD-037 - Restricted fields must be absent from unauthorized read models.

MOD-038 - Search indexes must include tenant/company ownership and permission filters.

MOD-039 - A dashboard value must link back to the permitted source record or explain that drill-down is restricted.

16. Audit and Observability

Every important action must produce a structured audit record.

- actor and acting role
- tenant and company
- support-access or impersonation state
- domain, record type and record ID

- action and status transition
- before and after values where appropriate
- timestamp, request ID and correlation ID
- result and error classification

Audit logs are not a replacement for application logs, metrics and traces. The system needs all four: business audit, technical logs, metrics and distributed tracing.

17. API and Data Model Requirements

- Commands and queries use separate service methods even if they share the same HTTP framework.
- Every operational table includes tenant_id and owning_company_id where applicable.
- Foreign-key validation prevents cross-tenant linking.
- Optimistic concurrency or version fields protect important records from lost updates.
- Money uses Decimal and explicit currency.
- Events use versioned schemas.
- Sensitive data selectors are centralized.
- Background jobs process events, notifications, indexing and read-model updates.

18. Migration from Current Repository

- 1 Create a module ownership map for all existing Prisma models.
- 2 Identify duplicated source fields and designate the authoritative owner.
- 3 Replace direct Prisma writes in pages/actions with application services.
- 4 Create central policy evaluation and migrate module permission checks.
- 5 Add tenant/company validation to every relation used by create and update commands.
- 6 Introduce transactional outbox and event consumers.
- 7 Move dashboard/profile aggregation into permission-safe query services.
- 8 Convert financial Float fields to Decimal through controlled migrations.
- 9 Split oversized orchestration services without changing external behavior.
- 10 Add idempotency, audit consistency and lifecycle transition tests.

Migration priority

Authorization, tenant isolation and source-of-truth defects must be fixed before adding more cross-module automation.

19. Acceptance Criteria

- AC-01 - No important frontend or server action writes directly to Prisma outside an approved application service.
- AC-02 - Each core business field has one documented owning domain.
- AC-03 - A dashboard cannot fetch Finance or HR fields that the viewer is not permitted to access.
- AC-04 - Changing a role or membership invalidates stale authorization.
- AC-05 - Cross-tenant and unauthorized cross-company record links are rejected.

- AC-06 - Approvals and status transitions are enforced by backend lifecycle rules.
- AC-07 - Submitting the same payment, task creation or approval command twice does not create duplicates.
- AC-08 - Contract approval produces durable events and all required downstream effects.
- AC-09 - Client and contractor profiles display source Finance data without duplicating payment records.
- AC-10 - Every important action appears in the central audit trail with actor, company and correlation ID.
- AC-11 - Read models can be rebuilt from authoritative records and events.
- AC-12 - Unit and integration tests cover authorization, events, idempotency, transactions and invalid transitions.

20. Full Stack Implementation Checklist

Architecture

- ☐ Domain ownership map approved.
- ☐ Application-service boundaries defined.
- ☐ Central policy engine adopted.
- ☐ Event and outbox conventions documented.

Backend

- ☐ Direct Prisma writes removed from UI actions.
- ☐ All referenced IDs validated by tenant/company.
- ☐ Lifecycle transitions centralized.
- ☐ Idempotency implemented for high-risk commands.

Database

- ☐ Money converted to Decimal.
- ☐ Ownership and company indexes added.
- ☐ Unique constraints support business idempotency.
- ☐ Audit and outbox persistence made durable.

Frontend

- ☐ Pages use permission-safe query models.
- ☐ Linked records open source pages.
- ☐ Status actions reflect backend transition rules.
- ☐ Activity timeline uses shared event/audit format.

QA and Security

- ☐ Negative authorization tests added.
- ☐ Cross-tenant write tests added.
- ☐ Duplicate command tests added.
- ☐ Event replay and retry tests added.
- ☐ Audit completeness tests added.

Final engineering rule

Build modules as domains inside one operating system - not as separate applications connected by copied data and frontend shortcuts.